

# PROJECT CASE STUDIES

## Leon Infant B.

*Full-Stack & AI Engineering — Portfolio Documentation*

Clear, practical, and problem-focused — a record of what was built, why, and what it taught me.

# Table of Contents

|                                       |   |
|---------------------------------------|---|
| <b>Table of Contents</b> .....        | 2 |
| <b>The Problem</b> .....              | 3 |
| <b>Why I Built It</b> .....           | 3 |
| <b>My Role</b> .....                  | 3 |
| <b>Technical Decisions</b> .....      | 3 |
| <b>Challenges</b> .....               | 3 |
| <b>What I Learned</b> .....           | 4 |
| <b>Outcome</b> .....                  | 4 |
| <b>The Problem</b> .....              | 4 |
| <b>Why I Built It</b> .....           | 4 |
| <b>My Role</b> .....                  | 5 |
| <b>Technical Decisions</b> .....      | 5 |
| <b>Challenges</b> .....               | 5 |
| <b>What I Learned</b> .....           | 5 |
| <b>Outcome</b> .....                  | 6 |
| <b>Problem</b> .....                  | 6 |
| <b>Why I Built It</b> .....           | 6 |
| <b>My Role</b> .....                  | 6 |
| <b>Technical Decisions</b> .....      | 6 |
| <b>Challenges</b> .....               | 7 |
| <b>What I Learned</b> .....           | 7 |
| <b>Outcome</b> .....                  | 7 |
| <b>The Problem</b> .....              | 7 |
| <b>Why I Built It</b> .....           | 8 |
| <b>My Role</b> .....                  | 8 |
| <b>Technical Decisions</b> .....      | 8 |
| <b>The Hardest Part</b> .....         | 8 |
| <b>What I Learned</b> .....           | 8 |
| <b>Where It Stands Now</b> .....      | 9 |
| <b>About Me</b> .....                 | 9 |
| <b>Contact</b> .....                  | 9 |
| <b>Before (Generic AI Copy)</b> ..... | 9 |
| <b>After (Edited Version)</b> .....   | 9 |
| <b>Why I Changed It</b> .....         | 9 |

# 1. Restaurant Ordering & Billing System

*Role: Analytics & Dashboard Developer — Team Project (Internship)*

## The Problem

Most small restaurants still run on manual processes — order slips, handwritten bills, and no real tracking of daily sales. This leads to order mistakes, slow service, and owners with no clear picture of daily performance. There was no single place where ordering, billing, and sales data came together.

## Why I Built It

This was part of my internship. I wanted hands-on experience solving an actual business problem rather than another isolated tutorial project. A restaurant management system touches almost every part of a real application — frontend, backend, database, APIs, analytics, and background processing — so I could see how all of it fits together, not just one slice of it.

## My Role

I worked on a team of four and owned the analytics and dashboard side of the system:

- Building the admin dashboard
- Creating the analytics APIs
- Calculating revenue data
- Integrating charts for revenue trends and order statistics
- Building CSV export for reports
- Setting up Redis caching
- Implementing Celery background jobs for report generation

Day to day, this meant designing the dashboard flow, wiring frontend components to backend APIs, handling data visualization, and ensuring the analytics features held up against real data from the rest of the system.

## Technical Decisions

Redis for caching — dashboard data such as revenue summaries and stats gets requested repeatedly. Hitting the database every time is wasteful, so frequently accessed analytics data was cached to keep response times down.

Celery for background jobs — tasks like generating reports and exporting CSVs don't need to happen within the request-response cycle. Running them as background jobs kept the application responsive instead of making users wait on slower operations.

A React-compatible charting library for visualizing revenue trends and order statistics on the dashboard.

The goal wasn't chasing the "best" tool for each piece — it was picking what fit the stack the team had already agreed on, and understanding why each piece was there: how caching, background jobs, APIs, and visualization work together in a running application, not just in theory.

## Challenges

The biggest challenge wasn't a specific bug — it was integration. The dashboard was built against sample data first, since the backend (owned by other team members) was developed in parallel. When it came time to connect to real API responses, the data structures didn't match what had been assumed, requiring adjustments to the dashboard components.

This was resolved by talking directly with the teammates who owned orders and billing, testing their endpoints directly, and reworking components against the actual response shapes instead of guessing. It also required properly understanding how Redis and Celery fit into the wider application flow, not just one corner of it.

## What I Learned

The technical side — connecting dashboards to APIs, using caching and background jobs, getting services to talk to each other — was only part of it. The bigger shift was realizing that shipping a feature isn't just about writing code for one module. It's about understanding the whole flow, communicating with the team building the other pieces, and breaking a large problem into parts that actually fit together.

Before this project, success was mostly measured by "does my code work." Afterward, the focus shifted to how each part affects the rest of the system and the people using it.

## Outcome

As a team, we delivered a working system with authentication, menu management, ordering, billing, and analytics modules integrated together. My contribution was the analytics and dashboard layer — revenue calculations, charts, CSV export, caching, and background processing — all working against live data from the rest of the app.

The real outcome wasn't a metric — it was going through the full cycle of building a feature that has to integrate with other people's work, in a system built to solve an actual business problem.

## 2. AI-Based Insurance Query System

*Role: Solo Developer — End-to-End Build*

### The Problem

Insurance is confusing by design. Policies are full of dense terms and conditions, and most people don't have a quick way to get a straight answer to a simple question. Instead, they either dig through long documents themselves or wait on an agent. The goal was to shortcut that — giving people a fast, understandable explanation without needing to become an insurance expert first.

### Why I Built It

Rather than building another AI demo that just shows a model responding to prompts, the aim was to apply AI to a real problem and go through the full process of building a usable application around it — not just the AI part, but the interaction design, the prompt engineering, and the plumbing that connects everything. Insurance was a good fit because the pain point is real and the value of a clear answer is obvious.

## My Role

Built solo, end to end:

- Designing the UI in React
- Building the application flow — how a question moves from the user to the AI and back
- Writing the backend logic in Flask
- Integrating the OpenAI API and doing the prompt design
- Setting up MongoDB for application data
- Connecting all of these pieces into one working system

## Technical Decisions

React for the frontend — needed something flexible for building an interactive Q&A interface.

Flask (Python) for the backend — lightweight, and Python has strong support for working with LLM APIs, keeping the AI integration straightforward.

OpenAI API for the AI layer — training or fine-tuning a model from scratch wasn't practical for this project. Using an existing model made it possible to focus time on the application flow and response quality instead of model infrastructure.

MongoDB for storage — flexible enough for the application data without needing a rigid schema upfront.

The stack wasn't chosen to impress anyone — it was chosen to focus on what actually mattered: how the user asks a question, and how good the answer is.

## Challenges

The hardest part wasn't the integration — it was getting the AI to give genuinely useful answers. Early responses were too generic because insurance questions need context to answer well. This required reworking the prompts: giving clearer format instructions, telling the model to explain things in plain language, and keeping it focused on insurance topics instead of drifting into generic answers.

Ambiguous user questions were another problem. Response quality turned out to be directly tied to how well-structured the input was, so a good part of the work became shaping the interaction flow, not just the prompt text itself.

On top of that, there was the normal friction of getting frontend, backend, and an external API to talk to each other reliably — nothing dramatic, but it forced careful thinking about handling external services properly in a real app, not just a script.

## What I Learned

Building an AI application is not just "call an API, get a response." The real work is in understanding the problem, designing prompts that actually produce useful output, and building a good experience around the AI so the answers land the way they're supposed to. This also meant learning to test and iterate based on what the output actually looked like, instead of assuming the first prompt or flow was good enough.

## Outcome

The result was a complete, working AI-based insurance query system — one that shows how an LLM can solve a real, practical problem rather than just being a chatbot demo. It provided hands-on experience with full-stack AI integration and has become one of the projects referenced when explaining how software engineering and AI come together.

## 3. Pizza Delivery App

*A full-stack food ordering application built with the MERN stack, covering the complete flow from browsing the menu to placing an order.*

### Problem

Traditional food ordering — especially over the phone or through disorganized systems — comes with real friction: delays, limited menu visibility, no way to track an order, and no easy way for restaurants to manage incoming orders. Customers need a simple way to browse items, place orders, and track purchases; restaurants need a better way to manage what's coming in without the chaos. The goal was a system connecting both sides through a smoother, more organized process.

### Why I Built It

This wasn't built for a client or a real restaurant — it was a learning project, intended to move past simple CRUD apps toward something closer to a real-world application. A pizza delivery system checked that box: it needed product management, user accounts, cart handling, orders, payments, and constant frontend-backend communication — all the pieces that show up in actual e-commerce applications. The goal was to understand how these modules work together in a complete system, not just build them in isolation.

### My Role

Built solo, handling both frontend and backend:

- Designing the UI and building out React components
- Building backend APIs with Node.js and Express
- Connecting the application to the database
- Managing the full flow — from product selection, to cart, to order placement

Working across the whole stack required understanding how a full-stack application is actually structured, not just how each piece works on its own.

### Technical Decisions

MERN stack — Frontend: React · Backend: Node.js with Express · Database: MongoDB · Styling: Tailwind CSS.

MongoDB made sense because the data — menu items, users, orders — is fairly flexible. A document-based database made it easier to structure and update that kind of data without fighting a rigid schema.

State management was handled with React's built-in state and context, not Redux. The app wasn't big enough to justify Redux — adding it would have meant more boilerplate for no real benefit. Plain React state kept things simple and easy to follow.

Tailwind CSS allowed the UI to be built and adjusted quickly without writing and maintaining a lot of custom CSS.

The common thread across these choices: match the tools to the size of the project, not to what's popular — simpler architecture, fewer moving parts, easier to reason about.

## Challenges

The hardest part wasn't any single feature — it was keeping the frontend and backend in sync. Cart updates, order creation, and general state management got messy early on: the frontend sometimes showed data that didn't match what was stored in the database, leading to unpredictable behavior.

This was fixed by restructuring how API calls were made — checking backend responses properly and only updating frontend state after confirming a successful response from the server, instead of assuming the update would go through. It reinforced that building a full-stack app isn't just about building a frontend and a backend separately — the real work is in designing how they talk to each other reliably.

## What I Learned

Before this project, the focus was mostly on getting individual features working. Building this shifted that focus toward the full user journey — how modules connect, and how one design decision (like when to update state) affects the rest of the system.

On the technical side, this provided real, hands-on experience with API design, state management, database structuring, and frontend-backend communication — not just in theory, but from running into the problems that come with getting them to work together.

## Outcome

This is a portfolio/demo project, not a deployed production app. It was built to understand the full MERN development workflow end to end — from the interface down to the database. The main outcome wasn't a metric or a user count; it was practical experience building a complete application and understanding how real-world web apps are actually structured underneath.

## 4. FlyRank AI Portfolio Website

*A case study in building a portfolio that shows how I think, not just what I built*

### The Problem

Most developer portfolios are just screenshots and tech-stack lists. They show what someone built but not how they think — what problems they actually solved, what tradeoffs they made, and why. A recruiter or hiring manager is left guessing at the reasoning behind the work. The goal was a portfolio that closed that gap — one that showed the problem-solving process, not just a project gallery.

## Why I Built It

Writing case studies in a doc, or using a template builder, would only describe the relevant skills, not demonstrate them. Since the focus was frontend and AI engineering, it made more sense to build something that was itself proof of that skill — a real interactive application combining UI development, AI-powered features, and thought-through UX. It also provided practice at taking an idea all the way from concept to a finished product, not just shipping isolated features.

## My Role

A solo project — ownership covered planning the structure, designing the frontend, building the React components, implementing the AI interactions, and refining the UX throughout. No one else touched the code or the decisions.

## Technical Decisions

React — a component-based structure made it easier to build and reuse interactive UI pieces without the app turning into spaghetti.

Tailwind CSS — allowed fast design iteration and consistent styling without maintaining a separate stylesheet architecture.

Streaming AI responses — this was the decision that took the most time. Instead of showing a spinner and waiting for the full AI response, the frontend renders content incrementally as it arrives — token by token, with loading states and smooth appending. A spinner-and-wait approach would have worked technically, but feels slow and disconnected; streaming makes the interface feel alive while the response is still generating.

Deployment — a modern frontend hosting platform was used for simple, repeatable deploys, enabling continued iteration without friction.

The overall principle behind the stack: pick tools that allow fast iteration and a clean architecture, not tools that add complexity for its own sake.

## The Hardest Part

Getting the streaming experience right. Receiving partial text from an API is one thing — updating the UI smoothly without it feeling janky or triggering unnecessary re-renders is another. This required careful thought about when state should update, when to show a loading indicator versus real content, and how to append new text without the component tripping over itself.

That process taught a bigger lesson: building an AI feature isn't just wiring up an API call. Most of the real work is in designing around latency — making something feel responsive while it's still working in the background.

## What I Learned

The technical lesson was about state management and rendering performance under streaming conditions. The bigger lesson was about process — planning the user flow before writing code, especially for anything involving AI, where response time and uncertainty are part of the experience by default.

Rebuilding this today, the AI response architecture — error states, different response types, scalability — would be designed from day one instead of added in along the way. This project shifted the underlying role from someone who makes features work to someone who makes AI interactions feel reliable and predictable to the person using them.

## Where It Stands Now

It's live and part of the active portfolio — the site people see when evaluating frontend and AI engineering work. It isn't built for traffic or metrics; it's built to demonstrate technical decision-making. The current version includes the full frontend experience, the AI interaction flow, streaming responses, and the loading/rendering work described above.

## 5. About Me & Contact

### About Me

I am a Computer Science Engineering student focused on building full-stack applications and AI-powered products.

I enjoy solving real problems through software, learning new technologies, and turning ideas into working applications. My projects focus on understanding user needs, making practical technical decisions, and continuously improving as a developer.

### Contact

Interested in collaborating, discussing projects, or sharing ideas? Let's connect.

**LinkedIn:** [linkedin.com/in/leon-infant-b-34b6bb328](https://www.linkedin.com/in/leon-infant-b-34b6bb328)

## 6. Before vs. After — Bio Copy Edit

### Before (Generic AI Copy)

*"I am a passionate and results-driven developer who builds innovative, cutting-edge solutions using modern technologies to solve complex problems."*

### After (Edited Version)

*"I build full-stack and AI-powered applications by focusing on real problems, practical solutions, and the decisions behind the technology I use."*

### Why I Changed It

The edited version removes generic phrases like "passionate," "results-driven," and "cutting-edge." It focuses on the actual work, the type of applications built, and the approach to solving problems.